

Porównanie algorytmów poszukiwania najkrótszych ścieżek w grafie o nieujemnych krawędziach

Ewa Kasprzak

7 stycznia 2025

1 Introduction

1.1 Dijkstra

Zasada Działania

1. Inicjalizacja:

- Wektor `dist` jest inicjalizowany maksymalnymi wartościami odległości, z wyjątkiem węzła startowego, którego odległość jest ustawiona na 0.
- Wykorzystanie kolejki priorytetowej (`minHeap`) do efektywnego wybierania węzłów o najmniejszej odległości.

2. Relaksacja:

- Dla każdego węzła u pobieranego z kolejki, aktualizowane są odległości do jego sąsiadów v , jeśli znajdzie się krótsza ścieżka przez u .

3. Zakończenie:

- Proces trwa, aż kolejka będzie pusta, co oznacza przetworzenie wszystkich węzłów.

Złożoność

Dla struktury `std::priority_queue` złożoności operacji są następujące:

- Wstawienie elementu (operacja `push`): $\mathcal{O}(\log n)$, gdzie n to liczba elementów w kolejce.
- Usunięcie elementu (operacja `pop`): $\mathcal{O}(\log n)$.

Algorytm Dijkstry składa się z następujących kroków:

- **Inicjalizacja:** Tworzymy wektor `dist`, który przechowuje najkrótsze odległości od wierzchołka początkowego do pozostałych wierzchołków grafu. Tworzenie tego wektora ma złożoność $\mathcal{O}(n)$, gdzie n to liczba wierzchołków.
- **Dodanie początkowego wierzchołka do kolejki:** Wstawiamy pierwszy wierzchołek do min-heap, co kosztuje $\mathcal{O}(\log n)$.
- **Główna pętla (while):** Pętla działa do momentu, gdy min-heap jest pusty. Dla każdego wierzchołka:
 - Usuwamy wierzchołek z kolejki (`pop`), co ma złożoność $\mathcal{O}(\log n)$.
 - Przechodzimy przez wszystkie sąsiadujące wierzchołki. Dla każdej krawędzi, jeżeli można zaktualizować odległość, wstawiamy sąsiedni wierzchołek do kolejki (`push`), co również kosztuje $\mathcal{O}(\log n)$.

Każda operacja `push` i `pop` jest wykonywana maksymalnie n razy (po jednym razie dla każdego wierzchołka), więc złożoność tych operacji to $\mathcal{O}(n \log n)$.

Dla każdej krawędzi w grafie (łącznej liczbie m krawędzi), jeśli aktualizujemy odległość wierzchołka, to wykonujemy operację `push` w min-heap, co kosztuje $\mathcal{O}(\log n)$. Zatem całkowity koszt związany z krawędziami to $\mathcal{O}(m \log n)$.

Całkowita złożoność algorytmu Dijkstry, uwzględniając zarówno wierzchołki, jak i krawędzie, wynosi:

$$\mathcal{O}((n + m) \log n)$$

gdzie:

- n to liczba wierzchołków,
- m to liczba krawędzi w grafie.

Złożoność pamięciowa algorytmu Dijkstry jest zależna od rozmiaru grafu oraz przechowywanych odległości. Całkowita złożoność przestrzenna wynosi $\mathcal{O}(n + m)$, ponieważ musimy przechowywać:

- graf w postaci list sąsiedztwa (złożoność $\mathcal{O}(m)$),
- wektor odległości `dist` (złożoność $\mathcal{O}(n)$).

1.2 Dial

Ta implementacja algorytmu Dial oblicza najkrótsze ścieżki od węzła startowego do wszystkich innych węzłów w grafie, z uwzględnieniem maksymalnej wagi krawędzi.

Zasada Działania

1. Inicjalizacja:

- Wektor `dist` inicjalizowany jest maksymalnymi wartościami, z wyjątkiem węzła startowego, którego odległość ustawiona jest na 0.
- Tworzone są kubelki (`buckets`) o liczbie równej `max.weight + 1`, gdzie każda wartość odległości `dist` będzie przypisywana do odpowiedniego kubelka.

2. Przetwarzanie Kubelków:

- Węzły są przetwarzane w kolejności rosnącej według aktualnych kubelków odległości.
- Dla każdego węzła `u`, aktualizowane są odległości do jego sąsiadów `v`, jeśli znajdzie się krótsza ścieżka przez `u`.

3. Zakończenie:

- Proces trwa, aż wszystkie kubelki są puste, co oznacza przetworzenie wszystkich węzłów.

Złożoność

- **Czasowa:** $O(E + V \cdot C)$, gdzie C to maksymalna waga krawędzi, a W to liczba kubelków.
- **Pamięciowa:** $O(V + E + C)$

1.3 RadixHeap

Ta implementacja algorytmu radix Dijkstra oblicza najkrótsze ścieżki od węzła startowego do wszystkich innych węzłów w grafie.

Zasada Działania

1. Inicjalizacja:

- Wektor `dist` jest inicjalizowany maksymalnymi wartościami, z wyjątkiem węzła startowego, którego odległość ustawiona jest na 0.
- Węzły są początkowo dodawane do zbioru `not_visited`.
- Tworzone są kubelki (`buckets`), które będą przechowywać węzły w zależności od odległości od węzła startowego.

2. Aktualizacja Kubelków:

- Proces przetwarzania węzłów odbywa się poprzez przenoszenie ich między kubelkami.

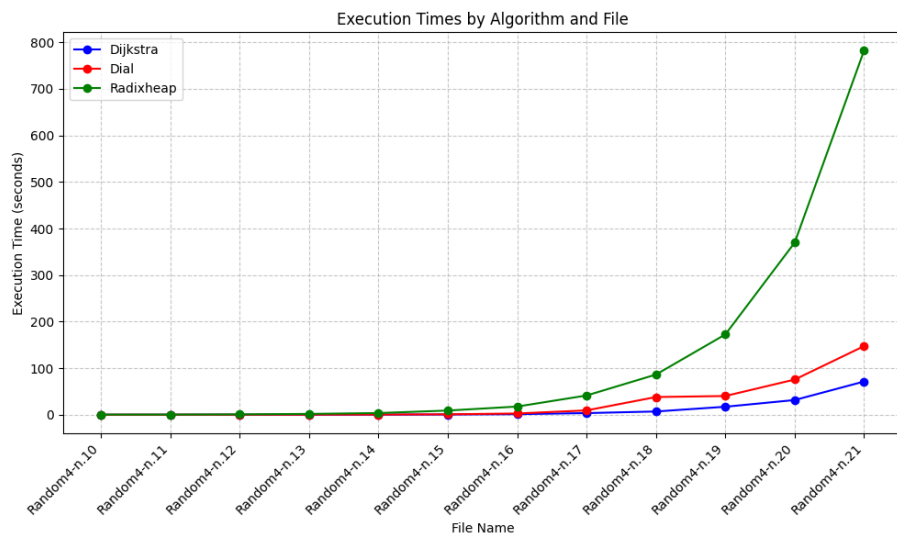
- Dla każdego węzła u , aktualizowane są odległości do jego sąsiadów v , jeśli znajdzie się krótsza ścieżka przez u .

3. Zakończenie:

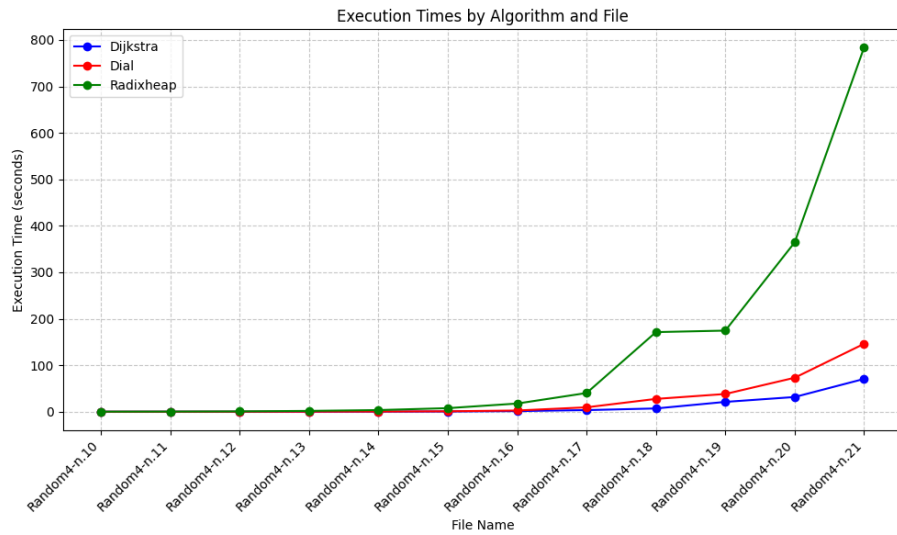
- Proces trwa, aż zbiór `not_visited` będzie pusty, co oznacza przetworzenie wszystkich węzłów.

Złożoność

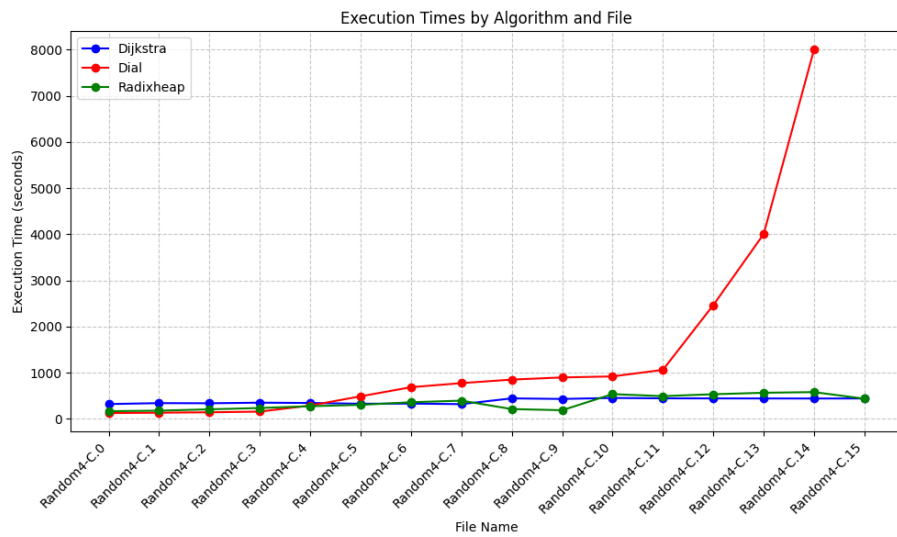
- **Czasowa:** $O(E + V \log V)$
- **Pamięciowa:** $O(V + E + C)$, gdzie C to liczba kubelków.



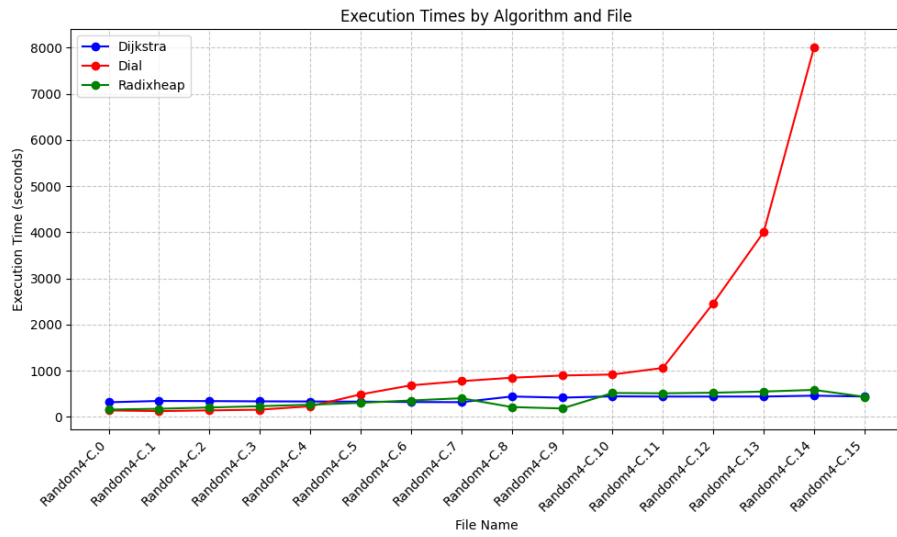
Rysunek 1: Random4-n min



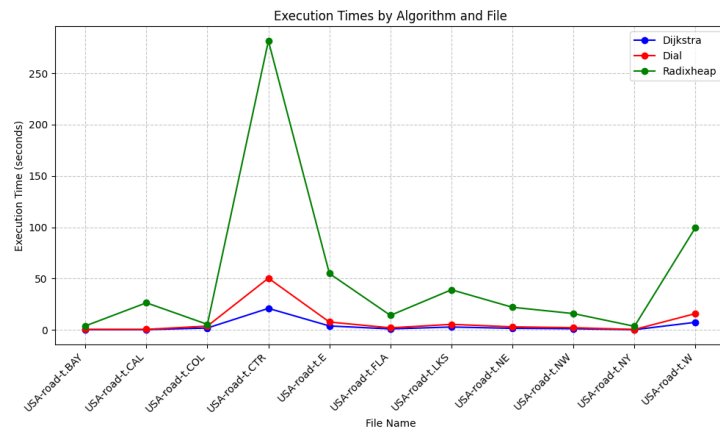
Rysunek 2: Random4-n avg



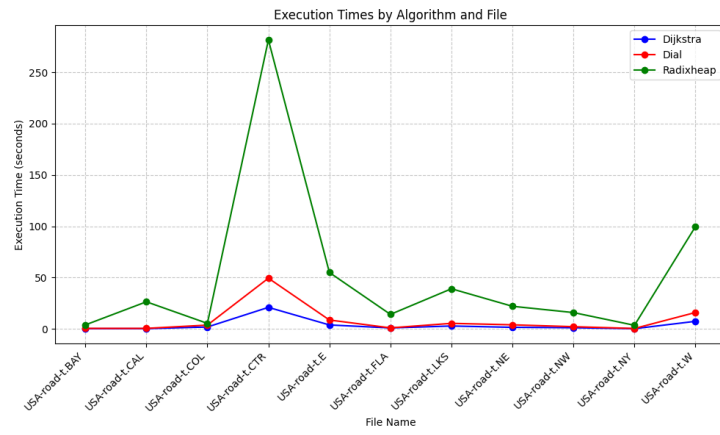
Rysunek 3: Random4-C min



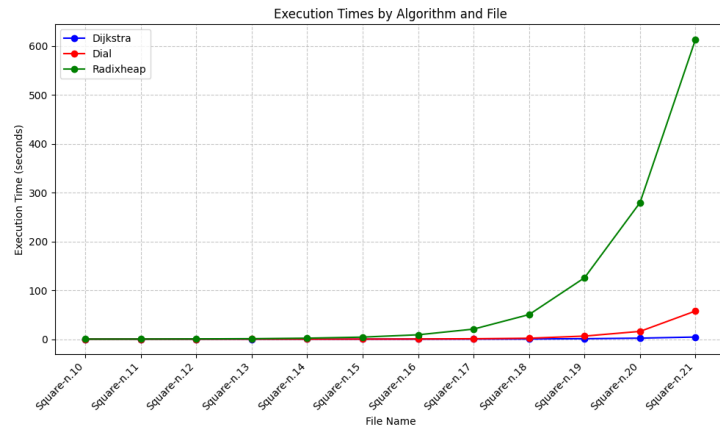
Rysunek 4: Random4-C avg



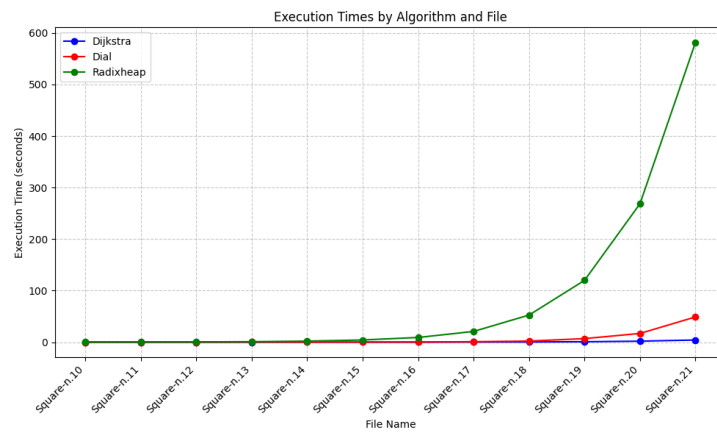
Rysunek 5: USA-t min



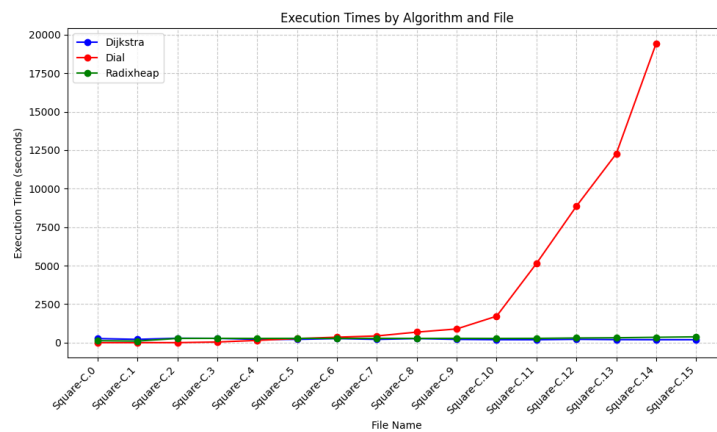
Rysunek 6: USA-t avg



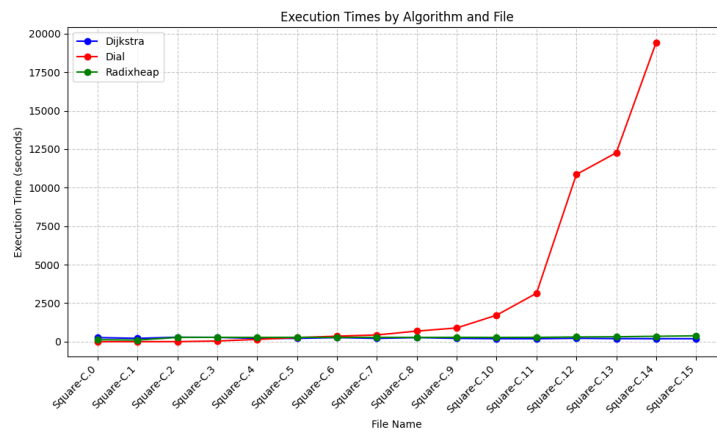
Rysunek 7: Square-n min



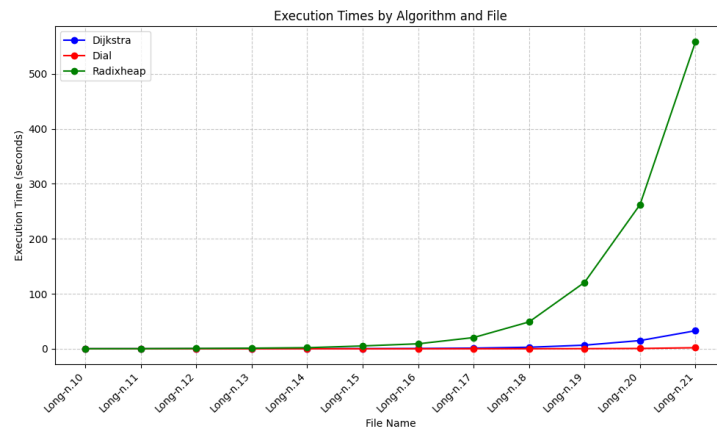
Rysunek 8: Square-n avg



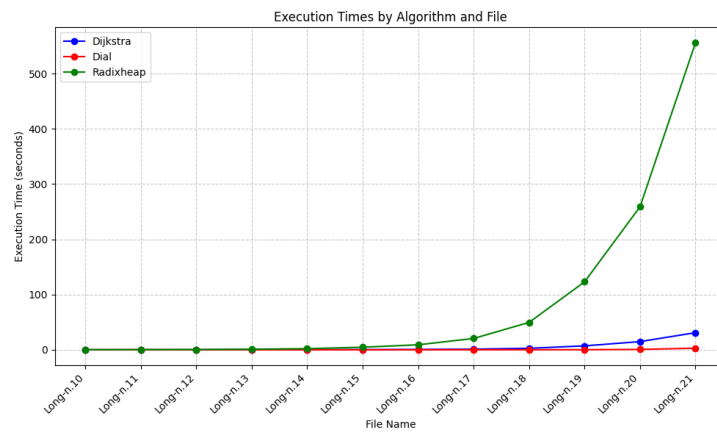
Rysunek 9: Square-C min



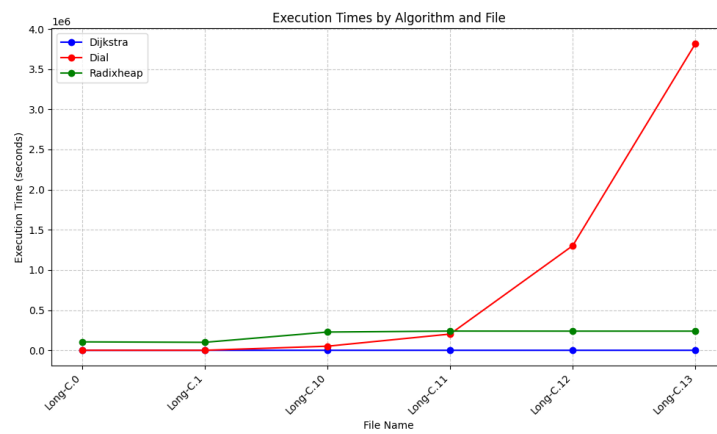
Rysunek 10: Square-C avg



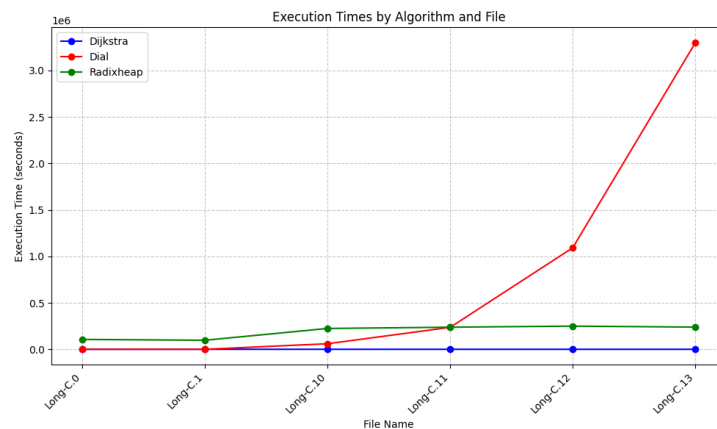
Rysunek 11: Long-n min



Rysunek 12: Long-n avg



Rysunek 13: Long-C min



Rysunek 14: Long-C avg

2 Wnioski

- Dla grafów o małych wartościach C najlepszym wyborem jest algorytm **Diala**
- Algorytmem, który najgorzej sobie radzi jest algorytm **RadixHeap**, wynika to najprawdopodobniej z dużej stałej ukrytej w notacji duże O , która pojawiła się jako efekt implementacji algorytmu.
- Najoptymalniejszym algorytmem jest algorytm **Dijkstry**.

Algorytm **Dijkstry** nadal pozostaje najlepszym wyborem w ogólnym przypadku, jednak specyficzne struktury grafu mogą przyczynić się do większej efektywności algorytmu Diala (struktury o ograniczonych wagach krawędzi) lub opartego na strukturze RadixHeap (duże grafy o ograniczonych wagach krawędzi).